Aula 15 – Integração Front-end e Back-end

Introdução

Em aplicações web modernas, o **Front-end** e o **Back-end** trabalham juntos para fornecer uma experiência completa ao usuário. O Front-end é responsável pela interface visual e interação com o usuário, enquanto o Back-end processa as informações, realiza validações, acessa bancos de dados e retorna respostas.

Nesta aula, estudaremos:

- Recebimento de dados de formulários;
- Validação de dados;
- Processamento no servidor.

---

1. O que é Integração Front-end e Back-end?

A integração ocorre quando os dados inseridos pelo usuário em uma página web são enviados para um servidor para processamento.

Fluxo básico

1. Usuário preenche um formulário.
2. Front-end envia os dados.
3. Back-end recebe os dados.
4. O servidor valida as informações.
5. O servidor processa os dados.
6. Uma resposta é enviada ao usuário.

Exemplo

Um usuário realiza um cadastro:

- Nome: João Silva
- E-mail: joao@email.com
- Senha: 123456

Esses dados são enviados ao servidor para serem armazenados no banco de dados.

---

2. Formulários HTML

Os formulários são utilizados para coletar informações dos usuários.

Estrutura básica

```
<form action="/cadastro" method="post">

 <label>Nome:</label>
 <input type="text" name="nome">

 <label>E-mail:</label>
 <input type="email" name="email">

 <button type="submit">Enviar</button>

</form>
```

Elementos comuns

Elemento	Função
input text	Entrada de texto
input email	E-mail
input password	Senha
textarea	Texto longo
select	Lista suspensa
button	Botão

---

3. Métodos GET e POST

Os formulários podem enviar dados utilizando principalmente dois métodos.

Método GET

Envia os dados pela URL.

Exemplo:

https://site.com/pesquisa?produto=notebook

Vantagens

- Simples;
- Pode ser compartilhado.

Desvantagens

- Dados ficam visíveis na URL;
- Não recomendado para informações sensíveis.

Método POST

Envia os dados no corpo da requisição.

Exemplo:

<form method="post">

Vantagens

- Mais seguro;
- Permite envio de grandes volumes de dados.

Desvantagens

- Não pode ser facilmente compartilhado.

---

4. Recebimento dos Dados no Back-end

Após o envio do formulário, o servidor recebe as informações.

Exemplo em PHP

```
<?php

$nome = $_POST["nome"];
$email = $_POST["email"];
```

```
echo "Nome: " . $nome;
echo "Email: " . $email;

?>
```

Exemplo em Node.js (Express)

```
app.post('/cadastro', (req, res) => {

  const nome = req.body.nome;
  const email = req.body.email;

  res.send("Cadastro recebido");

});
```

Exemplo em C#

```
[HttpPost]
public IActionResult Cadastro(Usuario usuario)
{
  return Ok("Dados recebidos");
}
```

---

5. Validação de Dados

Validação é o processo de verificar se os dados recebidos são válidos antes do processamento.

Ela pode ocorrer em:

- Front-end;
- Back-end.

O ideal é utilizar ambos.

---

6. Validação no Front-end

A validação no Front-end melhora a experiência do usuário.

Exemplo HTML

```
<input
  type="email"
  required>
```

O atributo:

required

impede o envio do formulário vazio.

Exemplo JavaScript

```
function validarFormulario() {

  let email = document.getElementById("email").value;

  if(email === "") {

     alert("Informe o e-mail");

     return false;
  }

  return true;
}
```

---

7. Validação no Back-end

Mesmo que exista validação no Front-end, ela pode ser burlada.

Por isso o servidor deve validar novamente.

Exemplo PHP

```
if(empty($_POST["email"])) {

  echo "E-mail obrigatório";

}
```

Exemplo C#

```
if(string.IsNullOrEmpty(usuario.Email))
```

```
{
  return BadRequest("Email obrigatório");
}
```

---

8. Tipos de Validação

Campo obrigatório

Verifica se o usuário preencheu o campo.

```
if(nome === "")
```

Tamanho mínimo

Verifica quantidade mínima de caracteres.

```
if(senha.length < 6)
```

Formato de e-mail

Verifica se o e-mail possui formato válido.

```
const regex = /\S+@\S+\.\S+/;
```

Número válido

Verifica se o valor é numérico.

```
isNaN(valor)
```

Faixa de valores

Verifica limites permitidos.

```
idade >= 18
```

---

9. Processamento no Servidor

Após a validação, os dados podem ser processados.

Exemplos:

- Cadastro de usuários;
- Atualização de registros;
- Emissão de notas fiscais;
- Geração de relatórios;
- Envio de e-mails.

Exemplo de Cadastro

Dados recebidos

Nome: João
Email: joao@email.com

Processamento

1. Validar dados;
2. Verificar duplicidade;
3. Salvar no banco;
4. Retornar sucesso.

---

10. Comunicação com APIs

Muitas aplicações modernas enviam dados utilizando APIs.

Exemplo com Fetch

```
fetch('/api/usuarios', {

  method: 'POST',

  headers: {
    'Content-Type': 'application/json'
  },

  body: JSON.stringify({
    nome: 'João',
```

```
      email: 'joao@email.com'
  })

})
.then(response => response.json())
.then(data => console.log(data));
```

---

11. Respostas do Servidor

Após o processamento, o servidor envia uma resposta.

Sucesso

```
{
 "status": "sucesso"
}
```

Erro

```
{
 "status": "erro",
 "mensagem": "E-mail inválido"
}
```

---

12. Boas Práticas

Sempre validar no Back-end

A validação do Front-end pode ser desabilitada.

Sanitizar entradas

Remover caracteres perigosos.

Utilizar HTTPS

Protege os dados durante a transmissão.

Não confiar nos dados recebidos

Todo dado enviado pelo usuário deve ser considerado potencialmente inválido.

Exibir mensagens claras

Ajuda o usuário a corrigir erros rapidamente.

Resumo da Aula

Nesta aula aprendemos:

- Como os formulários enviam dados ao servidor;
- Diferenças entre GET e POST;
- Como receber dados no Back-end;
- Validação no Front-end;
- Validação no Back-end;
- Processamento das informações;
- Comunicação utilizando APIs.

A integração entre Front-end e Back-end é fundamental para o funcionamento de aplicações web, garantindo que os dados sejam recebidos, validados e processados corretamente.

Exercícios

Exercício 1

Explique a diferença entre Front-end e Back-end.

Exercício 2

Descreva o fluxo completo de envio de dados de um formulário até o processamento no servidor.

Exercício 3

Qual a principal diferença entre os métodos GET e POST?

Exercício 4

Crie um formulário HTML contendo:

- Nome
- E-mail
- Senha
- Botão Enviar

Exercício 5

Por que a validação deve ser feita também no Back-end?

Exercício 6

Escreva uma validação em JavaScript que impeça o envio de um formulário com o campo nome vazio.

Exercício 7

Cite três tipos de validação que podem ser realizadas em um formulário.

Exercício 8

Explique o que significa processamento no servidor.

Exercício 9

Qual a função da API em uma aplicação web moderna?

Exercício 10

Liste cinco boas práticas relacionadas à integração entre Front-end e Back-end e explique a importância de cada uma.